Scalable Tree Ensemble Proximities in Python

Adrien Aumon ADRIEN.AUMON@UMONTREAL.CA
Université de Montréal; Mila – Quebec AI Institute

Guy Wolf GUY.WOLF@UMONTREAL.CA
Université de Montréal; Mila – Quebec AI Institute

Kevin R. Moon KEVIN.MOON@USU.EDU
Utah State University

Jake S. Rhodes RHODES@STAT.BYU.EDU
Brigham Young University

Abstract

Tree ensemble methods such as Random Forests naturally induce supervised similarity measures through their decision tree structure, but existing implementations of proximities derived from tree ensembles typically suffer from quadratic time or memory complexity, limiting their scalability. In this work, we introduce a general framework for efficient proximity computation by defining a family of Separable Weighted Leaf-Collision Proximities. We show that any proximity measure in this family admits an exact sparse matrix factorization, restricting computation to leaf-level collisions and avoiding explicit pairwise comparisons. This formulation enables low-memory, scalable proximity computation using sparse linear algebra in Python. Empirical benchmarks demonstrate substantial runtime and memory improvements over traditional approaches, allowing tree ensemble proximities to scale efficiently to datasets with hundreds of thousands of samples on standard CPU hardware. Our code is available at `https://github.com/jakerhodes/RF-GAP-Python`.

Keywords: Tree Ensemble Proximities, Sparse Matrix Factorization, Near-linear Complexity

1 Introduction

Beyond their widespread use in classification and regression, Random Forests (RFs, [2, 3]) and other tree ensemble methods such as Extremely Randomized Trees (ExtraTrees, [7]) or Gradient Boosted Trees (GBTs, [6]) have gained significant interest for their natural ability to induce pairwise similarity measures based on the partitioning structure of their decision trees. The notion of forest proximity between two observations was originally introduced by Leo Breiman as the proportion of trees in which both observations fall into the same leaf [3]. Since tree splits are optimized with respect to a supervised objective, these proximities encode a meaningful supervised similarity measure that accounts for variable importance relative to auxiliary information.

Forest-based proximities have since been leveraged in a wide range of applications, including hierarchical structure discovery [10, 4], guided data exploration [13, 1], prototype identification [16], tabular classification via RF-induced graph neural networks [5], time-series classification with proximity forests [15] and supervised manifold alignment [12]. Despite their versatility, forest-based proximities are rarely used at scale due to the quadratic time or memory complexity of existing implementations. Rhodes et al. [13] hypothesized

©2026 Aumon et al..

License: CC-BY 4.0, see `https://creativecommons.org/licenses/by/4.0/`.

that for a given training point, leaf-based proximities could be computed by comparing it only to points that co-occur with it in the same leaves across trees, thereby avoiding explicit comparisons with all training samples. However, this idea was left as future work.

In this paper, we formalize this insight by introducing a family of leaf-based proximities termed *Separable Weighted Leaf-Collision Proximities*, showing that these proximities can be written as a product of two sparse matrices. By restricting computation to leaf-level collisions, our method avoids redundant pairwise comparisons and maintains both efficiency and low memory use. We validate our theoretical results by comparing the runtime and memory requirements of our proposed high-level Python implementation with traditional approaches. Our experiments demonstrate that the proposed method scales efficiently to datasets with hundreds of thousands of points on standard CPU hardware, while conventional implementations struggle under the same conditions.

2 Methods

2.1 Factorizable leaf-collision proximities

While standard RF proximities are based solely on leaf co-occurrence, Rhodes et al. [14] showed that weighted leaf co-occurrences better capture the intrinsic geometry and generalization properties of a trained Random Forest. To extend our scalable proximity framework to proximity definitions with arbitrary weighting schemes, we introduce the following family of proximities.

Definition 1 (Weighted Leaf-Collision Proximities) *Let $\mathcal{T} = \{t \mid t = 1, \ldots, T\}$ be the tree indices of an ensemble. For any observation $\mathbf{x}_i$, let $\ell_i(t)$ denote the unique leaf in tree t containing $\mathbf{x}_i$. We define the Weighted Leaf-Collision Proximity (WLCP) matrix $P \in \mathbb{R}^{N \times N}$ by its entries,*

$$P_{ij} = \sum_{t=1}^{T} w_{ijt} \cdot \mathbb{I}(\ell_i(t) = \ell_j(t)), \tag{1}$$

*where $w_{ijt} \in \mathbb{R}$ is a weight scale representing the importance of a collision between samples i and j in tree t. Moreover, a WLCP is said to be **separable** if there exist sample-local weights q_{it} and w_{jt} such that the pairwise weight factorizes as:*

$$w_{ijt} = q_{it} \cdot w_{jt}. \tag{2}$$

Eq. 2 implies that query and weight importances are determined solely by the individual sample and tree-specific metadata, independent of the paired observation. This separability is the fundamental property that enables the sparse matrix factorization established in the following proposition. Indeed, by ensuring q and w depend only on a single sample index, the contribution of each tree t can be decomposed into the outer product of two independent, sample-wise vectors.

While one could theoretically compute separable WLCPs by looping over samples and accumulating contributions from visited leaves, such a row-by-row strategy is poorly suited for high-level languages like Python. Even when the asymptotic complexity of an explicit loop matches that of a sparse matrix operation, the resulting interpreter overhead and

lack of cache optimization create a significant performance bottleneck. To address this limitation, we express proximity construction as a global sparse matrix-matrix product. This formulation is mathematically equivalent to the traditional accumulation method, but shifts the computational workload to optimized, compiled sparse linear algebra routines [17, 8].

Proposition 1 (Sparse matrix factorization) *Any separable WLCP matrix P can be exactly factorized as $P = QW^\top$, where $Q, W \in \mathbb{R}^{N \times |\mathcal{K}|}$ and $\mathcal{K}$ is the set of all unique leaves in the ensemble of trees. Furthermore, Q and W are strictly row-sparse matrices with **at most** T non-zero entries per row.*

Proof See Appendix A. ■

Proposition 1 shows that separable weights are enough to get a sparse factorization. By fulfilling this condition, the memory footprint is reduced to a linear function of the sample size, effectively bypassing the traditional quadratic bottleneck. In practice, this benefit is further amplified because most weighting schemes use ensemble metadata (e.g. leaf mass, out-of-bag status, or tree importance) that is intrinsically available after the training phase. Since these components are essentially "built-in" to the tree ensemble structure, the resulting sparse matrix factorization serves as a scalable framework for proximity-based analysis.

The versatility of this framework is demonstrated in the following corollary, where we show that the sparse $QW^\top$ matrix decomposition can recover several standard proximity definitions by simply identifying their respective sample-local weighting factors.

Corollary 1.1 *The original RF proximity matrix P^{orig} [2], the RF-GAP proximity matrix P^{GAP} [14], and the GBT proximity matrix P^{GBT} [16] are separable WLCPs and thus admit a sparse $QW^\top$ factorization.*

Proof See Appendix A. ■

2.2 Symmetry slicing

2.3 Complexity analysis

The sparse matrix decomposition $P = QW^\top$ established in Proposition 1 provides a computationally efficient path for both calculating and storing proximities. By leveraging the intrinsic sparsity of the tree ensemble structure, we move from a quadratic $\mathcal{O}(N^2T)$ bottleneck to a near-linear complexity. Indeed, this formulation implicitly restricts computation to colliding points, namely observations that fall into the same leaves, while maintaining low-memory footprint through the proximity computation pipeline due to the high sparsity of the intermediate component matrices (Q, W) and the resulting pairwise proximity matrix P.

This subsection presents the Big O upper bounds under standard structural assumptions for common tree ensemble approaches. Specifically, we assume balanced tree structures where the tree height h is logarithmic relative to the number of samples ($h \approx \log N$).

Time complexity. The total time complexity for constructing the proximity matrix from scratch is $\mathcal{O}(T \cdot N \log N \cdot p + TN(h + \ell))$. This near-linear scaling is derived from three primary sequential stages:

- Ensemble training ($\mathcal{O}(T \cdot N \log N \cdot p)$): Training scales as $\mathcal{O}(T \cdot N \log N \cdot p)$ ([11, 9]). This represents the cost for balanced trees where $p \ll N$ features are evaluated per split. This generalizes to RFs and ExtraTrees (often parallelized) and GBT variants (sequential).

- Leaf encoding and weight assignment ($\mathcal{O}(TNh)$): To populate the sparse matrices Q and W, every sample traverses each tree to determine its leaf index $\ell_i(t)$. Simultaneously, the query and weight importances (q_{it}, w_{jt}) are retrieved from tree metadata. For an average tree height $h \approx \log N$, this joint operation scales as $\mathcal{O}(TNh)$. Because these importance values are ensemble-intrinsic (e.g., node mass, OOB status), they are assigned during the traversal with $\mathcal{O}(1)$ overhead, ensuring complete scalability.

- Sparse proximity calculation ($\mathcal{O}(TN\ell)$): The product $P = QW^\top$ is computed via optimized sparse kernels. For a query sample i in tree t, a non-zero contribution only occurs for the $\ell \ll N$ samples residing in the same leaf (mean leaf size). This scales as $\mathcal{O}(TN\ell)$. With sufficiently deep trees, ℓ is a small constant, keeping this stage effectively linear.

The additive complexity accounts for the sequential nature of the pipeline. Since p and ℓ are typically negligible compared to N, the overall complexity is effectively near-linear.

Space complexity. The primary bottleneck of traditional implementations is the $\mathcal{O}(N^2)$ storage of the dense similarity matrix. Our sparse factorization avoids this by storing only the non-zero interactions. As proved in Proposition 1, each row of Q and W contains at most T non-zero entries. Stored in Compressed Sparse Row (CSR) format, the number of non-zero elements is $\text{nnz}(Q) + \text{nnz}(W) \leq 2NT$, which is strictly linear $\mathcal{O}(NT)$. The resulting matrix P is also generated and stored sparsely. For a query sample i, the number of non-zero interactions is bounded by the number of trees T multiplied by the average leaf size ℓ:

$$\text{nnz}(P) \approx N \times (T \cdot \ell)$$

Combining these stages, the total space complexity is $\mathcal{O}(NT(1 + \ell))$. Since T and ℓ are typically small constants relative to N, the overall memory footprint is effectively $\mathcal{O}(N)$. This enables proximity-based analysis for $N > 10^5$ on standard CPU hardware, where a dense N^2 representation would require ≈ 75 GB of RAM.

3 Benchmark

To evaluate the practical efficiency of the $QW^\top$ factorization, we compared our sparse proximity framework with the baseline implementation introduced by Rhodes et al. [14]. Benchmarks were conducted on the Fashion-MNIST dataset [18] using subset sizes $N \in \{k \cdot 10,000 \mid k = 1, \ldots, 7\}$. Experiments were performed on a MacBook Pro equipped with an Apple M4 Pro chip (12-core CPU: 8 performance and 4 efficiency cores) and 24

GB of unified memory. For each N, we report the mean CPU wall-clock time (Fig. 1a) and peak RAM usage (Fig. 1b) over five independent trials. These measurements include both Random Forest training and the construction of the original and RF-GAP proximity matrices using both the naive pairwise and our sparse implementations. Additionally, we display the scaling exponents m, derived from power-law fits, adjacent to their corresponding curves in Fig. 1a.

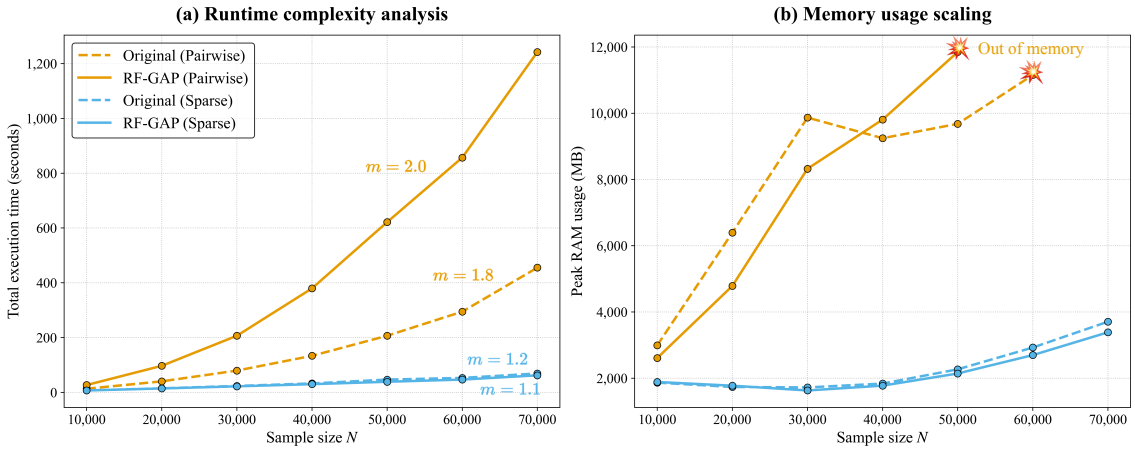


Figure 1: Empirical complexity and scalability analysis on the Fashion-MNIST dataset comparing naive pairwise implementations with our sparse framework. **(a)** Runtime analysis demonstrates near-linear scaling ($m \approx 1.1$) for the sparse method, while pairwise implementations exhibit quadratic growth ($m \approx 2.0$). **(b)** Memory usage scaling highlights the efficiency of the sparse approach; pairwise curves are truncated at $N = 50,000$ and $N = 60,000$ due to process termination resulting from memory exhaustion.

In Figure 1a, the execution time for the naive pairwise implementations exhibits quadratic growth, characterized by an empirical scaling exponent of $m \approx 2.0$. In contrast, the sparse implementations scale near-linearly with $m \approx 1.1$. This efficiency allows the sparse method to complete the full proximity computation (including the ensemble training phase) in a few seconds. For the full data, the naive RF-GAP implementation requires more than 20 minutes to complete the same task, demonstrating a significant gap in temporal efficiency.

As depicted in Figure 1b, the memory usage for the original and RF-GAP pairwise implementations does not strictly follow a quadratic curve. These baselines were designed as "semi-sparse" implementations, generating proximity vectors one row at a time to avoid the $O(N^2)$ memory bottleneck. While this approach reduces the final storage footprint, the runtime remains quadratic because the algorithm must still explicitly detect collisions for every possible sample pair. Still, these pairwise methods eventually failed due to memory exhaustion and memory swapping at about $N = 50,000$ and $N = 60,000$. Our proposed approach avoids these limitations by leveraging the global tree structure to compute proximities through a single sparse matrix multiplication. By bypassing explicit pairwise com-

parisons, our method required less than 4 GB of RAM for the full Fashion-MNIST dataset, representing a negligible footprint on modern hardware.

Finally, these results show that our RF-GAP implementation is consistently more efficient in both execution time and memory usage. This validates the "collision-only" proximity strategy; by focusing exclusively on OOB/In-Bag collisions, the resulting matrix is inherently sparser than the original formulation. This allows the algorithm to scale effectively to large datasets, whereas naive versions become resource-intensive and fail to use the inherent benefits of the RF-GAP framework.

4 Conclusion

Despite their ability to encode label-informed relationships, proximities derived from tree ensembles typically suffer from quadratic time and space complexity in the number of samples. We show that proximity matrix construction via explicit pairwise comparisons can be avoided by using a sparse matrix factorization that implicitly targets only nonzero interactions in the leaf space. We implement this framework in high-level Python and demonstrate its scalability in practice. Compared to existing implementations, our approach achieves near-linear time and space complexity, allowing tree-based proximities to scale to large datasets.

References

[1] A. Aumon, S. Ni, M. Lizotte, G. Wolf, K. R. Moon, and J. S. Rhodes. Random Forest Autoencoders for Guided Representation Learning. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, Oct. 2025.

[2] L. Breiman. Random forests. *Machine learning*, 45(1):5–32, 2001.

[3] L. Breiman and A. Cutler. Manual on setting up, using, and understanding random forests v4.0. Technical report, University of California, Berkeley, Department of Statistics, 2003.

[4] L. Duvaux, Q. Geissmann, K. Gharbi, J.-J. Zhou, J. Ferrari, C. M. Smadja, and R. K. Butlin. Dynamics of Copy Number Variation in Host Races of the Pea Aphid. *Molecular Biology and Evolution*, 32(1):63–80, Jan. 2015. ISSN 1537-1719, 0737-4038. doi: 10.1093/molbev/msu266.

[5] S. Farokhi, H. Chen, K. Moon, and H. Karimi. Advancing Tabular Data Classification with Graph Neural Networks: A Random Forest Proximity Method. In *2024 IEEE International Conference on Big Data (BigData)*, pages 7011–7020, Dec. 2024. doi: 10.1109/BigData62323.2024.10825972.

[6] J. H. Friedman. Greedy Function Approximation: A Gradient Boosting Machine. *The Annals of Statistics*, 29(5):1189–1232, 2001. ISSN 0090-5364.

[7] P. Geurts, D. Ernst, and L. Wehenkel. Extremely randomized trees. *Machine Learning*, 63(1):3–42, Apr. 2006. ISSN 1573-0565. doi: 10.1007/s10994-006-6226-1.

[8] F. G. Gustavson. Two Fast Algorithms for Sparse Matrices: Multiplication and Permuted Transposition. *ACM Transactions on Mathematical Software*, 4(3):250–269, Sept. 1978. ISSN 0098-3500, 1557-7295. doi: 10.1145/355791.355796.

[9] K. Hassine, A. Erbad, and R. Hamila. Important Complexity Reduction of Random Forest in Multi-Classification Problem. In *2019 15th International Wireless Communications & Mobile Computing Conference (IWCMC)*, pages 226–231, June 2019. doi: 10.1109/IWCMC.2019.8766544.

[10] J. Lee. Patient-Specific Predictive Modeling Using Random Forests: An Observational Study for the Critically Ill. *JMIR Medical Informatics*, 5(1):e6690, Jan. 2017. doi: 10.2196/medinform.6690.

[11] G. Louppe. Understanding Random Forests: From Theory to Practice, June 2015.

[12] J. S. Rhodes and A. G. Rustad. Random Forest-Supervised Manifold Alignment. In *2024 IEEE International Conference on Big Data (BigData)*, pages 3309–3312. IEEE Computer Society, Dec. 2024. ISBN 979-8-3503-6248-0. doi: 10.1109/BigData62323.2024.10825663.

[13] J. S. Rhodes, A. Aumon, S. Morin, M. Girard, C. Larochelle, B. Lahav, E. Brunet-Ratnasingham, W. Zhang, A. Cutler, A. Zhou, D. E. Kaufmann, S. Zandee, A. Prat, G. Wolf, and K. R. Moon. Gaining Biological Insights through Supervised Data Visualization, Nov. 2023.

[14] J. S. Rhodes, A. Cutler, and K. R. Moon. Geometry- and Accuracy-Preserving Random Forest Proximities. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(9):10947–10959, Sept. 2023. ISSN 1939-3539. doi: 10.1109/TPAMI.2023.3263774.

[15] B. Shaw, J. Rhodes, S. F. Boubrahimi, and K. R. Moon. Forest Proximities for Time Series. In K. Arai, editor, *Intelligent Systems and Applications*, pages 588–598, Cham, 2026. Springer Nature Switzerland. ISBN 978-3-032-07109-5. doi: 10.1007/978-3-032-07109-5_40.

[16] S. Tan, M. Soloviev, G. Hooker, and M. T. Wells. Tree Space Prototypes: Another Look at Making Tree Ensembles Interpretable. In *Proceedings of the 2020 ACM-IMS on Foundations of Data Science Conference*, FODS '20, pages 23–34, New York, NY, USA, Oct. 2020. Association for Computing Machinery. ISBN 978-1-4503-8103-1. doi: 10.1145/3412815.3416893.

[17] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S. J. van der Walt, M. Brett, J. Wilson, K. J. Millman, N. Mayorov, A. R. J. Nelson, E. Jones, R. Kern, E. Larson, C. J. Carey, İ. Polat, Y. Feng, E. W. Moore, J. VanderPlas, D. Laxalde, J. Perktold, R. Cimrman, I. Henriksen, E. A. Quintero, C. R. Harris, A. M. Archibald, A. H. Ribeiro, F. Pedregosa, P. van Mulbregt, and SciPy 1.0 Contributors. SciPy 1.0: Fundamental algorithms for scientific computing in python. *Nature Methods*, 17:261–272, 2020. doi: 10.1038/s41592-019-0686-2.

[18] H. Xiao, K. Rasul, and R. Vollgraf. Fashion-MNIST: A Novel Image Dataset for Benchmarking Machine Learning Algorithms, Sept. 2017.

Appendix A. Proofs

Proof [Proposition 1] Let the columns of Q and W be indexed by $k \in \mathcal{K}$. Since each leaf k belongs to exactly one tree, let $t(k)$ be the tree index containing leaf k. We define the entries of Q and W as:

$$Q_{ik} = q_{i,t(k)} \cdot \mathbb{I}(\ell_i(t(k)) = k)$$
$$W_{jk} = w_{j,t(k)} \cdot \mathbb{I}(\ell_j(t(k)) = k)$$

The (i, j)-th entry of the product $QW^\top$ is obtained by grouping terms by tree. Let $\mathcal{L}(t)$ be the set of unique leaf indices in tree t. Then,

$$\begin{aligned}
(QW^\top)_{ij} &= \sum_{k\in\mathcal{K}} Q_{ik}W_{jk} = \sum_{t=1}^{T} \sum_{k\in\mathcal{L}(t)} Q_{ik}W_{jk} \\
&= \sum_{t=1}^{T} \sum_{k\in\mathcal{L}(t)} \big[q_{it}\,\mathbb{I}(\ell_i(t) = k)\big]\big[w_{jt}\,\mathbb{I}(\ell_j(t) = k)\big] \\
&= \sum_{t=1}^{T} q_{it}w_{jt} \sum_{k\in\mathcal{L}(t)} \mathbb{I}(\ell_i(t) = k)\,\mathbb{I}(\ell_j(t) = k) \\
&= \sum_{t=1}^{T} q_{it}w_{jt}\mathbb{I}(\ell_i(t) = \ell_j(t)) \\
&= \sum_{t=1}^{T} w_{ijt}\mathbb{I}(\ell_i(t) = \ell_j(t)) = P_{ij}.
\end{aligned}$$

The second-to-last step follows from the fact that for any tree t, a sample belongs to exactly one leaf. Thus, the product of indicators $\mathbb{I}(\ell_i(t) = k)\mathbb{I}(\ell_j(t) = k)$ is non-zero (equal to 1) if and only if both samples are in leaf k. Summing over all $k \in \mathcal{L}(t)$ yields 1 if they share any leaf in tree t, and 0 otherwise. Substituting Eq. 2 recovers the form in Eq. 1.

Finally, for any sample i and tree t, there is exactly one leaf $k \in \mathcal{L}(t)$ such that $\ell_i(t) = k$. Thus, in each tree-block of columns, row i has at most one non-zero entry (specifically the value q_{it}). Summing across all T trees, the total number of non-zero entries in rows $Q_{i,:}$ and $W_{j,:}$ is at most T:

$$\mathrm{nnz}(Q_{i,:}) = \sum_{t=1}^{T} \mathbb{I}(q_{it} \neq 0) \leq T, \qquad \mathrm{nnz}(W_{j,:}) = \sum_{t=1}^{T} \mathbb{I}(w_{jt} \neq 0) \leq T.$$

■

Proof [Corollary 1.1] We recover these measures by specific sample-local choices of q_{it} and w_{jt}:

- **Original RF proximities**: Substituting $q_{it} = \frac{1}{T}$ and $w_{jt} = 1$ into Eq. 1 yields the standard definition:
$$P_{ij}^{\text{orig}} = \frac{1}{T} \sum_{t=1}^{T} \mathbb{I}(\ell_i(t) = \ell_j(t)).$$

- **RF-GAP proximities** [14]: Let S_i be the set of indices for trees where $\mathbf{x}_i$ is Out-of-Bag (OOB). Let $c_j(t)$ be the multiplicity of sample j in tree t, and $|M_j(t)|$ be the leaf mass of the leaf containing sample j. Substituting $q_{it} = \frac{\mathbb{I}(t \in S_i)}{|S_i|}$ and $w_{jt} = \frac{c_j(t)}{|M_j(t)|}$ into Eq. 1 yields:
$$P_{ij}^{\text{GAP}} = \frac{1}{|S_i|} \sum_{t \in S_i} \frac{c_j(t) \cdot \mathbb{I}(\ell_i(t) = \ell_j(t))}{|M_j(t)|}.$$
As both functions depend only on their own sample index and the tree metadata, they satisfy the separability requirement of Definition 1.

- **GBT proximities** [16]: Substituting tree-level importance w_t into $q_{it} = \frac{w_t}{\sum_{k=1}^{T} w_k}$ and $w_{jt} = 1$ recovers the GBT formulation:
$$P_{ij}^{\text{GBT}} = \frac{\sum_{t=1}^{T} w_t \mathbb{I}(\ell_i(t) = \ell_j(t))}{\sum_{k=1}^{T} w_k}.$$

■